

Patrol Gameplay

1. Course Content

- Run the sample program, set the desired patrol route in the dynamic parameter adjuster, and then click start. The car will move according to the set patrol route. At the same time, the radar on the car will scan for obstacles within the set radar angle and obstacle detection distance. If an obstacle is found, it will stop and the buzzer will sound. If there are no obstacles, it will continue to patrol.

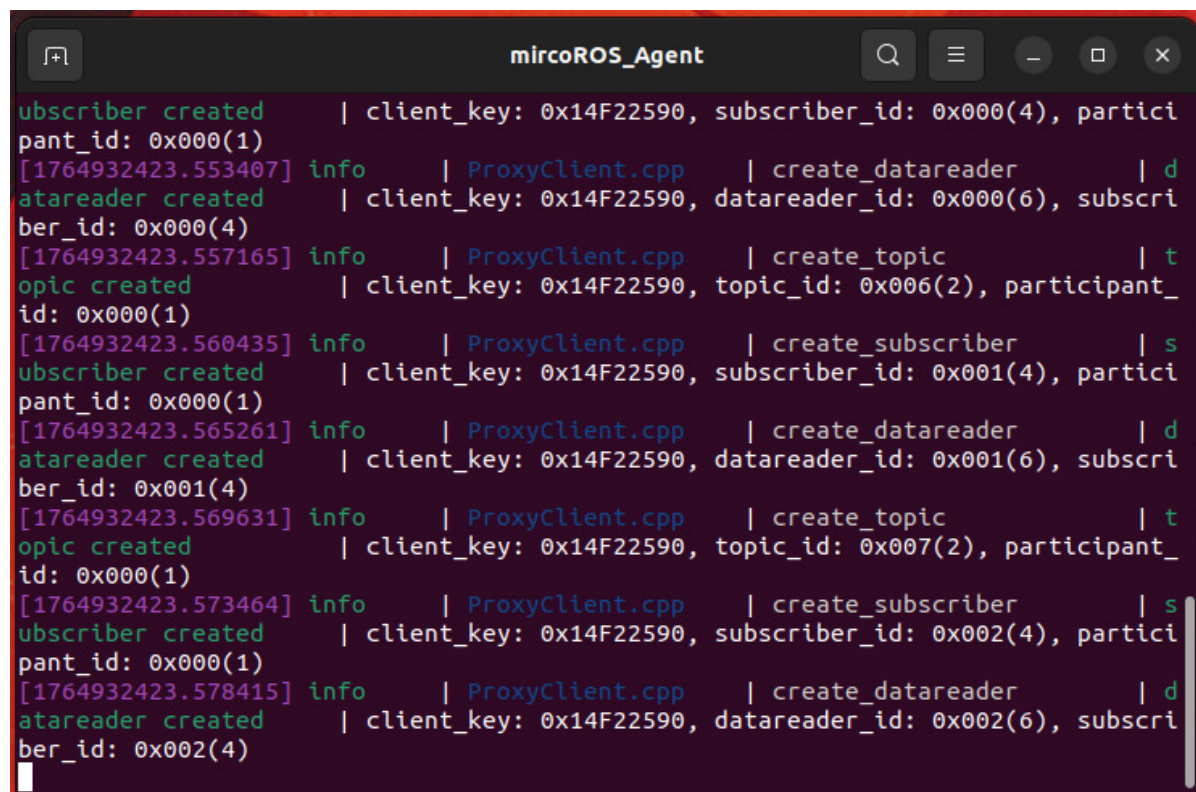
2. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



```
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

3. Run the Example

3.1 Start the Program

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

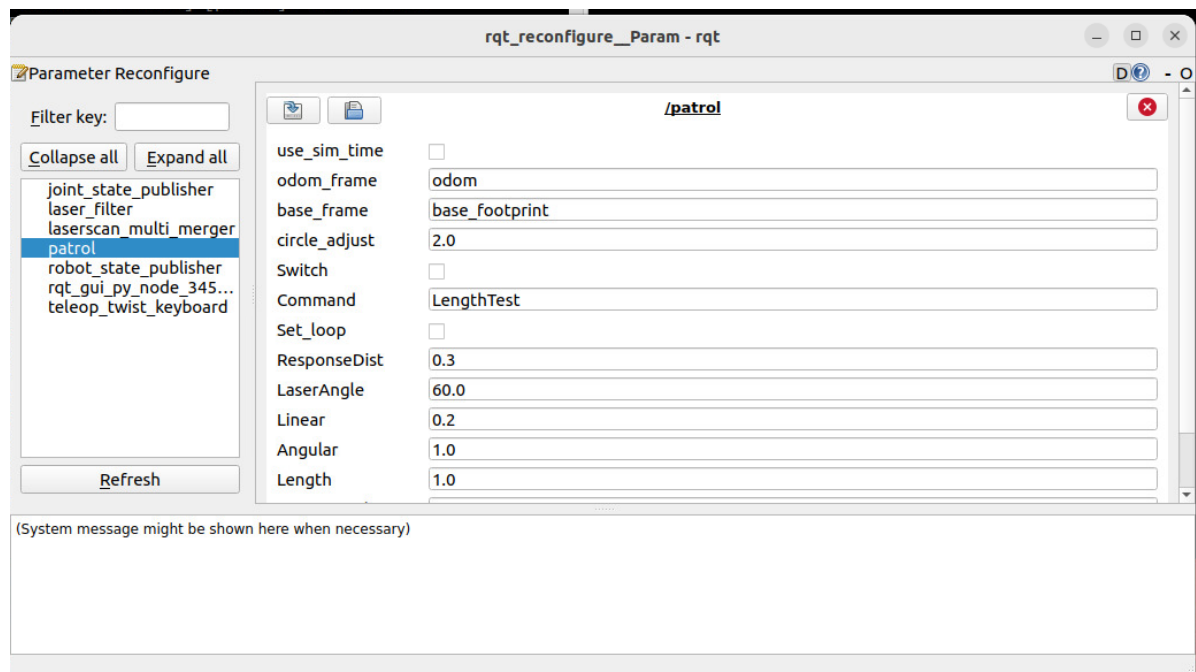
Open a terminal inside the container,

```
exec_m3
```

- Run the patrol program node:

```
ros2 launch m3_bringup patrol.launch.py
```

- Click the **patrol** node in the left options bar of the `rqt_reconfigure` panel. If there are no nodes on the left when you first start, click the **Refresh** button below.



[!TIP]

`rqt_reconfigure` panel parameter description:

- `odom_frame`: Name of the odometry coordinate frame.
- `base_frame`: Name of the base coordinate frame.
- `circle_adjust`: When the patrol route is a circle, this value can be used as a coefficient to adjust the size of the circle. See the code for details.
- `Switch`: Gameplay switch.
- `Command`: Patrol route, with the following options: [LengthTest]-straight line patrol, [Circle]-circle patrol, [Square]-square patrol, [Triangle]-triangle patrol.
- `Set_loop`: Switch to re-patrol. When set, it will continue to patrol in a loop according to the set route.
- `ResponseDist`: Obstacle detection distance.
- `LaserAngle`: Radar detection angle.
- `Linear`: Linear velocity magnitude.
- `Angular`: Angular velocity magnitude.
- `Length`: Distance for straight-line movement.

- `RotationTolerance`: Allowable tolerance for rotation error.
- `RotationScaling`: Proportional coefficient for rotation.
- **[Command]** is the set patrol route. Here, we take the straight-line patrol route as an example. The available patrol routes are described below.
- After setting the route in **[Command]**, click `Switch` to start patrolling. The terminal will print the following information:

```
[patrol-2] [INFO] [1765164370.602605186] [patrol]: LengthTest
[patrol-2] [INFO] [1765164370.604678533] [patrol]: distance: 0.0270851628480
32567
[patrol-2] [INFO] [1765164371.101304815] [patrol]: Switch True
[patrol-2] [INFO] [1765164371.102707260] [patrol]: LengthTest
[patrol-2] [INFO] [1765164371.104783647] [patrol]: distance: 0.0751071287024
9182
[patrol-2] [INFO] [1765164371.601953276] [patrol]: Switch True
[patrol-2] [INFO] [1765164371.603405194] [patrol]: LengthTest
[patrol-2] [INFO] [1765164371.605899706] [patrol]: distance: 0.1864800882978
9733
[patrol-2] [INFO] [1765164372.100411075] [patrol]: Switch True
[patrol-2] [INFO] [1765164372.101321312] [patrol]: LengthTest
[patrol-2] [INFO] [1765164372.102738669] [patrol]: distance: 0.3024302162352
4424
[patrol-2] [INFO] [1765164372.600518239] [patrol]: Switch True
[patrol-2] [INFO] [1765164372.601291384] [patrol]: LengthTest
[patrol-2] [INFO] [1765164372.602654403] [patrol]: distance: 0.4165993920396
192
[patrol-2] [INFO] [1765164373.101332018] [patrol]: Switch True
[patrol-2] [INFO] [1765164373.102755968] [patrol]: LengthTest
[patrol-2] [INFO] [1765164373.104203150] [patrol]: distance: 0.5255122298142
755
[patrol-2] [INFO] [1765164373.601116357] [patrol]: Switch True
[patrol-2] [INFO] [1765164373.602500273] [patrol]: LengthTest
[patrol-2] [INFO] [1765164373.604522642] [patrol]: distance: 0.6358100060346
081
[patrol-2] [INFO] [1765164373.604522642] [patrol]: distance: 0.6358100060346
081
```

- If there is an obstacle on the patrol path, the robot will stop and display **obstacles**.

```
273
[patrol-2] [INFO] [1765164426.606784070] [patrol]: obstacles
[patrol-2] [INFO] [1765164427.101254119] [patrol]: Switch True
[patrol-2] [INFO] [1765164427.102644307] [patrol]: LengthTest
[patrol-2] [INFO] [1765164427.104755415] [patrol]: distance: 0.4201478252386
0866
[patrol-2] [INFO] [1765164427.106456430] [patrol]: obstacles
[patrol-2] [INFO] [1765164427.600190744] [patrol]: Switch True
[patrol-2] [INFO] [1765164427.601009650] [patrol]: LengthTest
[patrol-2] [INFO] [1765164427.602968337] [patrol]: distance: 0.4200243627231
9627
[patrol-2] [INFO] [1765164427.604669384] [patrol]: obstacles
[patrol-2] [INFO] [1765164428.100545431] [patrol]: Switch True
[patrol-2] [INFO] [1765164428.101314992] [patrol]: LengthTest
[patrol-2] [INFO] [1765164428.102601529] [patrol]: distance: 0.4201504434069
9
[patrol-2] [INFO] [1765164428.103645818] [patrol]: obstacles
[patrol-2] [INFO] [1765164428.601229729] [patrol]: Switch True
[patrol-2] [INFO] [1765164428.602613998] [patrol]: LengthTest
[patrol-2] [INFO] [1765164428.604797044] [patrol]: distance: 0.4201396562012
752
[patrol-2] [INFO] [1765164428.606471337] [patrol]: obstacles
[patrol-2] [INFO] [1765164429.101270487] [patrol]: Switch True
[patrol-2] [INFO] [1765164429.102675524] [patrol]: LengthTest
[patrol-2] [INFO] [1765164429.104769991] [patrol]: distance: 0.4201245101829
2085
[patrol-2] [INFO] [1765164429.601197834] [patrol]: Switch True
[patrol-2] [INFO] [1765164429.602657433] [patrol]: LengthTest
```

4. Source Code Analysis

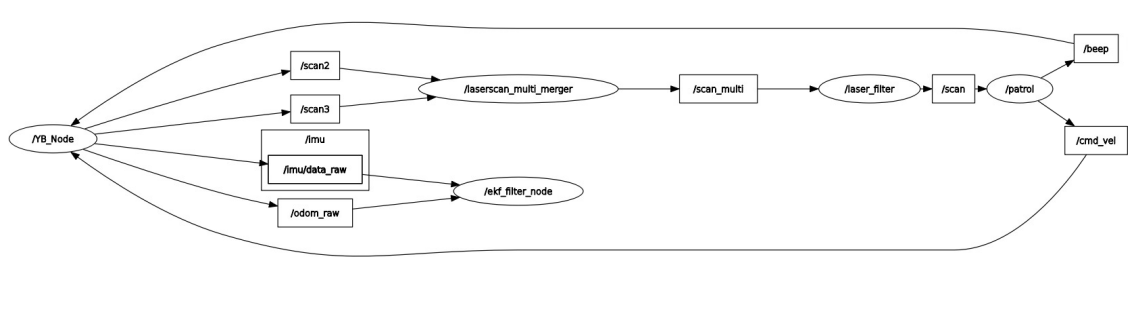
Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/patrol/patrol.py
```

4.1 View Node Relationship Graph

Open a terminal and enter the command:

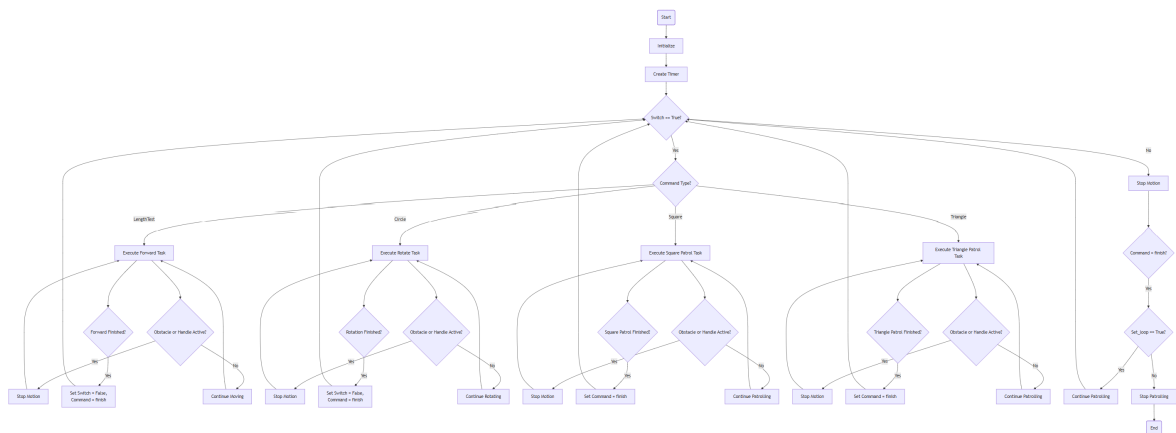
```
ros2 run rqt_graph rqt_graph
```



In the node relationship diagram above:

- **patrol** is the key node for implementing the patrol function. This node subscribes to the coordinate transformation between odometry `odom` and `base_footprint`, and subscribes to `/scan1` lidar data to determine if there are obstacles ahead.
- **YB_Node**: The chassis node, which publishes lidar data `/scan`, raw odometry data `/odom_raw`, and subscribes to the `/cmd_vel` topic to control the robot chassis movement via inverse kinematics.
- **ekf_filter_node**: An extended Kalman filter used to fuse raw odometry and filtered IMU data, publishing the fused `/odom` odometry data.
- **imu_filter**: The IMU filter node, which filters the raw IMU data `/imu/data_raw` and publishes the filtered `/imu/data` data.

4.2 Program Flowchart



4.3 Key Program

Below is an explanation of the core parts of the program:

Movement State Acquisition: Listens to the TF transform between `odom` and `base_footprint` to calculate the current XY coordinates and rotation angle.

Program Implementation: The `get_position` and `get_odom_angle` methods in the `YahboomCarPatrol` class.

```
def get_position(self):
    try:
        now = rclpy.time.Time()
        trans =
self.tf_buffer.lookup_transform(self.odom_frame, self.base_frame, now)
        return trans
    except (LookupException, ConnectivityException, ExtrapolationException):
        self.get_logger().info('transform not ready')
        raise
        return

def get_odom_angle(self):
    try:
        now = rclpy.time.Time()
        rot =
self.tf_buffer.lookup_transform(self.odom_frame, self.base_frame, now)
        #print("oring_rot: ",rot.transform.rotation)
        cac1_rot = PyKDL.Rotation.Quaternion(rot.transform.rotation.x,
rot.transform.rotation.y, rot.transform.rotation.z, rot.transform.rotation.w)
        #print("cac1_rot: ",cac1_rot)
        angle_rot = cac1_rot.GetRPY()[2]

    except (LookupException, ConnectivityException, ExtrapolationException):
        self.get_logger().info('transform not ready')
        return

    return angle_rot
```

Movement Control: The basic actions for the robot chassis movement. All patrol routes are combinations of straight lines and rotations.

Program Implementation: The `advancing` and `Spin` methods in the `YahboomCarPatrol` class.

```
def advancing(self, target_distance):
    self.position.x = self.get_position().transform.translation.x
    self.position.y = self.get_position().transform.translation.y
    move_cmd = Twist()
    self.distance = sqrt(pow((self.position.x - self.x_start), 2) +
                        pow((self.position.y - self.y_start), 2))
    self.distance *= self.LineScaling
    self.get_logger().info(f"distance: {self.distance}")
    self.error = self.distance - target_distance
    move_cmd.linear.x = self.Linear
    if abs(self.error) < self.LineTolerance :
```

```

        self.get_logger().info("stop")
        self.distance = 0.0
        self.pub_cmdvel.publish(Twist())
        self.x_start = self.position.x;
        self.y_start = self.position.y;
        self.Switch =
rclpy.parameter.Parameter('Switch', rclpy.Parameter.Type.BOOL, False)
        all_new_parameters = [self.Switch]
        self.set_parameters(all_new_parameters)
        return True
    else:
        if self.Joy_active or self.front_warning > 10:
            if self.moving == True:
                self.pub_cmdvel.publish(Twist())
                self.moving = False
                self.get_logger().info("obstacles")
            else:
                #print("Go")
                self.pub_cmdvel.publish(move_cmd)
            self.moving = True
            return False

def Spin(self, angle):
    self.target_angle = radians(angle)
    self.odom_angle = self.get_odom_angle()
    self.delta_angle = self.RotationScaling *
self.normalize_angle(self.odom_angle - self.last_angle)
    self.turn_angle += self.delta_angle
    # print("turn_angle: ", self.turn_angle)
    self.get_logger().info(f"turn_angle: {self.turn_angle}")
    self.error = self.target_angle - self.turn_angle
    self.get_logger().info(f"error: {self.error}")
    self.last_angle = self.odom_angle
    move_cmd = Twist()
    if abs(self.error) < self.RotationTolerance or self.Switch==False :
        self.pub_cmdvel.publish(Twist())
        self.turn_angle = 0.0
        '''self.Switch =
rclpy.parameter.Parameter('Switch', rclpy.Parameter.Type.BOOL, False)
        all_new_parameters = [self.Switch]
        self.set_parameters(all_new_parameters)'''
        return True
    if self.Joy_active or self.front_warning > 10:
        if self.moving == True:
            self.pub_cmdvel.publish(Twist())
            self.moving = False
            self.get_logger().info("obstacles")
        else:
            if self.Command == "Square" or self.Command == "Triangle":
                #move_cmd.linear.x = 0.2
                move_cmd.angular.z = copysign(self.Angular, self.error)
            elif self.Command == "Circle":
                length = self.Linear * self.circle_adjust / self.Length
                #print("length: ", length)
                move_cmd.linear.x = self.Linear
                move_cmd.angular.z = copysign(length, self.error)
                #print("angular: ", move_cmd.angular.z)

```

```
        '''move_cmd.linear.x = 0.2
        move_cmd.angular.z = copysign(2, self.error)'''
    self.pub_cmdvel.publish(move_cmd)
    self.moving = True
```